

Projet Compilation

Par Yunliang CAO numéro 273252

1. Compilation et exécution des tests

L'exécution de la commande make permet de générer l'ensemble des fichiers nécessaires au projet. En lançant make help, on obtient un résumé des commandes disponibles.

```
make all          : Build everything.
make test         : Run tests (TEST_FLAG can be -s|-t|-d|-h).
make tree         : Shortcut for 'make TEST_FLAG=-t test'.
make clean        : Remove build & report artifacts.
make help         : Show this help.
```

Il est possible de spécifier certains drapeaux de test via la variable **TEST_FLAG**. Par exemple, la commande **make TEST_FLAG=-s test** permet de lancer tous les tests du répertoire rep en affichant la **table des symboles** dans les fichiers résultats situés dans le dossier result. Les autres options incluent **-t** pour afficher l'**arbre syntaxique**, et **-d** pour activer le mode **debug**. **Attention** : utiliser directement **make -h** affichera l'aide de make lui-même dans tous les fichiers de sortie, ce qui est déconseillé.

La commande make test exécute l'ensemble des fichiers de test situés dans **test/**, et sauvegarde les résultats dans le dossier **rep/result**.

Il est également possible d'exécuter un fichier .tpc spécifique manuellement avec la commande suivante :

```
./bin/tpcc < exemple.tpc || ./bin/tpcc  exemple.tpc
```

Seuls les fichiers sans erreur (i.e. valides ou avec simples avertissements) génèrent un fichier .asm.

Si la redirection `<` est utilisée, le fichier assembleur généré portera automatiquement le nom `_anonymous.asm` dans le répertoire racine. Une exécution ultérieure de `make` produira alors un exécutable nommé `test_exec` à la racine du projet.

Si la redirection `<` n'est pas utilisée, le fichier `.asm` prendra le même nom que le fichier source (exemple.asm) et sera généré dans le dossier `nasm/`. Dans ce cas, la compilation de l'assembleur devra être effectuée manuellement.

2. Analyse Syntaxique

Par rapport au projet du semestre précédent, j'ai apporté plusieurs modifications à la partie analyse syntaxique, en particulier aux fichiers Flex et Bison.

Tout d'abord, j'ai corrigé certaines erreurs de structure dans le fichier Bison : il s'agissait de cas où des nœuds du graphe syntaxique étaient incorrectement liés — par exemple, des nœuds qui devaient être frères avaient été définis comme des nœuds enfants, ou encore certaines règles oublièrent d'inclure des identifiants essentiels. Bien que ces erreurs passaient les tests fournis auparavant, elles étaient incorrectes sur le plan sémantique et risquaient d'induire en erreur les phases ultérieures de l'analyse.

Ensuite, j'ai amélioré la reconnaissance des caractères de type `char`, en prenant en compte les cas particuliers liés aux caractères spéciaux, tels que `\n`. Ce problème m'est apparu clairement lorsque j'ai commencé à écrire les fonctions intégrées (builtins) : je me suis rendu compte qu'il était impossible de gérer correctement les retours à la ligne tant que ces cas n'étaient pas traités explicitement dans l'analyse lexicale.

```

CHAR      \'<(\\(x[0-9a-fA-F]+| [0-7]{1,3}|.|)| [^\\'\n]))\'

```

3 Table des symboles

La structure actuelle de la table des symboles n'a pas été définie entièrement dès le départ, mais s'est affinée progressivement au cours du développement. Au fil de l'implémentation, j'ai identifié certains cas où l'ajout d'un champ supplémentaire devenait nécessaire — soit pour des raisons de clarté, soit pour simplifier le traitement de certaines situations spécifiques. Cela dit, de tels ajustements sont restés rares : la structure globale avait été pensée dès le début et s'est révélée globalement stable.

De manière générale, la table est organisée en plusieurs couches : une table globale contenant les variables et les fonctions déclarées au niveau du programme, et pour chaque fonction, une table locale contenant ses paramètres et ses variables locales. Les symboles enregistrés incluent des informations telles que le nom, le type, le stockage (static ou non), la taille, l'offset dans la pile (pour les variables locales), ainsi qu'un label d'assemblage dans le cas des variables statiques.

```

1 typedef struct {
2     const char *type_name;    // Nom du type (ex : "int", "char", "void")
3 } TypeInfo;
4
5
6 /* Structure représentant une variable (locale, globale ou paramètre de fonction) */
7 typedef struct {
8     char    name[64];          // Nom de la variable
9     TypeInfo type;             // Type de la variable (int, char, void)
10    int     lineno, colno;      // Position dans le code source (ligne, colonne)
11    int     size;               // Taille mémoire allouée
12    int     offset;             // Décalage par rapport à %rbp
13    bool    used;               // Indique si la variable est effectivement utilisée
14    bool    is_static;          // Vrai si variable locale statique (restant en mémoire entre appels)
15    char    asm_label[256];     // Étiquette unique générée pour les variables statiques locales
16 } VarSym;
17
18
19 /* Table des variables (liste dynamique)*/
20 typedef struct {
21     VarSym *data;              // Tableau dynamique de symboles de variables (VarSym)
22     int     len;               // Nombre de variables actuellement stockées
23     int     cap;               // Capacité maximale actuelle du tableau (avant redimensionnement)
24     int     cur_offset;        // Offset courant pour l'allocation des variables locales (en négatif depuis %rbp)
25 } VarList;
26
27
28 /* Entrée de fonction (contient uniquement les symboles locaux et paramètres) */
29 typedef struct {
30     char    name[64];          // Nom de la fonction
31     TypeInfo ret_type;         // Type de retour (int, void...)
32     VarList params;            // Paramètres formels
33     VarList locals;           // Variables locales
34     int     lineno, colno;     // Position dans le code source
35     bool    used;              // Fonction utilisée ou non
36     bool    has_return;        // Présence explicite de 'return'
37     int     locals_bytes;      // Taille totale (en octets) de la pile locale, alignée
38 } FuncSym;
39
40
41 /* Table des symboles principale : variables globales + fonctions */
42 typedef struct {
43     VarList globals;          // Liste des variables globales
44     FuncSym *funcs;           // Tableau dynamique des fonctions définies dans le programme
45     int     func_len;          // Nombre de fonctions actuellement enregistrées
46     int     func_cap;          // Capacité actuelle du tableau 'funcs' (avant redimensionnement)
47     uint32_t n_error;          // Nombre total d'erreurs détectées
48     uint32_t n_warning;        // Nombre d'avertissements générés
49 } Symtab;
50
51

```

Mon objectif avec la table des symboles était de centraliser autant d'informations que possible, afin d'éviter de devoir consulter d'autres structures au cours des différentes phases du compilateur. Par exemple, même le nombre d'erreurs et d'avertissements détectés durant l'analyse sémantique est directement stocké dans la structure principale de la table.

Plusieurs choix de conception méritent d'être soulignés :

1. TypeInfo :

Ce type de structure avait été initialement conçu pour contenir des informations complètes sur les types — non seulement les types de base (**int**, **char**, **void**), mais potentiellement aussi des marqueurs indiquant qu'un symbole correspond à une fonction (**func**) ou d'autres caractéristiques plus complexes. Finalement, seuls les trois types de base ont réellement été utilisés, ce qui rend

TypeInfo redondant dans son état actuel. Cela dit, comme cette structure est déjà largement intégrée dans le code, je ne l'ai pas supprimée afin d'éviter une refactorisation trop lourde.

2. size, offset et cur_offset :

Ces champs sont principalement destinés à la génération de code assembleur. Un **char** est de taille 1 octet, tandis qu'un **int** occupe 4 octets. Les variables globales et statiques reçoivent un **offset** égal à 0, car elles ne sont pas placées dans la pile. En revanche, toutes les autres variables (locales ou paramètres passés par registre) reçoivent un offset négatif, qui descend par tranches de 8 octets. Ce n'est que lorsqu'un paramètre dépasse le sixième que des offsets positifs sont utilisés, car ces paramètres sont alors passés par la pile de l'appelant. Le champ **cur_offset** sert à suivre dynamiquement le prochain espace libre sur la pile lors de la construction des symboles locaux.

3. asm_label :

Ce champ est utilisé pour les variables statiques dans l'assemblage. Afin d'éviter les collisions de noms dans les cas de variables locales statiques ayant le même nom dans des fonctions différentes, chaque variable statique reçoit un label d'assemblage unique (généralement du type **NomDeFonction.Lf_nom**). Ce mécanisme permet ensuite de les référencer proprement via l'opérateur `[]`, ce qui simplifie considérablement l'accès mémoire dans le code assembleur généré.

Le fichier **symbol.c** est structuré autour de deux parcours successifs de l'arbre syntaxique abstrait (AST) :

Le premier parcours, effectué par la fonction **walk**, est dédié à la **construction de la table des symboles**. Durant cette phase, seuls les nœuds de déclaration sont réellement pris en compte. À chaque déclaration de variable

ou de fonction rencontrée, les informations pertinentes sont extraites et stockées dans les structures de symboles appropriées.

Le second parcours, réalisé par la fonction **check_usage**, est responsable de l'**analyse sémantique**. Il vérifie l'usage correct des symboles et signale les éventuelles erreurs ou incohérences. Bien que mes tests automatiques aient validé tous les cas avec un score de 100%, il est possible que certains messages soient en trop ou pas parfaitement conformes à l'intention du test (par exemple, un mauvais type d'erreur signalé).

Erreurs sémantiques détectées :

1. Redéclaration d'une variable globale
2. Conflit de nom entre une variable globale et une fonction (y compris les fonctions intégrées)
3. Redéfinition d'une fonction intégrée
4. Redéfinition d'une fonction utilisateur
5. Conflit entre nom de fonction et nom de variable globale
6. Noms de paramètres formels en double
7. Redéclaration de variable static locale
8. Redéclaration de variable locale normale
9. Utilisation illégale d'une expression vide (dans comparaisons, affectations, appels...)
10. Division par zéro (même si elle est traitée également dans la génération NASM par sécurité)
11. Mauvais usage des fonctions intégrées (get sans argument, put avec trop d'arguments ou arguments vides)
12. Appel à une fonction non déclarée
13. Appel incorrect à une fonction (mauvais nombre ou type d'arguments)
14. Utilisation d'un symbole non déclaré
15. Utilisation incorrecte d'un symbole (par exemple, utiliser une variable comme une fonction)
16. Expression vide dans une condition (if ou while)
17. Fonction void renvoyant une valeur
18. Fonction non void renvoyant une valeur vide
19. Fonction non void (hors main) sans instruction return
20. Absence de la fonction main
21. Retour non int pour la fonction main
22. Paramètres non void pour main

Chaque test du répertoire test/sem-error est conçu pour ne générer qu'une

seule erreur sémantique. Cependant, comme le programme poursuit l'analyse après une erreur (afin d'aider au débogage), il est possible que d'autres messages apparaissent à la suite. Ces erreurs supplémentaires peuvent généralement être ignorées.

Avertissements (warnings) générés :

1. Nom de paramètre formel identique à celui d'une fonction
2. Appel à putchar avec un argument non char
3. Conversion implicite de int vers char
4. Retour d'un type incorrect (par exemple : int au lieu de char)
5. Variable globale déclarée mais jamais utilisée
6. Fonction déclarée mais jamais appelée
7. Paramètre formel jamais utilisé
8. Variable locale jamais utilisée

Tous les tests du répertoire test/warn sont construits pour n'entraîner qu'un unique avertissement par fichier.

4.. Génération de code (NASM)

Je présente ici certaines décisions et opérations spécifiques que j'ai mises en place lors de la génération de code assembleur.

1. Gestion des registres par taille :

J'ai défini trois tableaux de registres correspondant respectivement aux versions 64 bits, 32 bits et 8 bits des six premiers registres d'arguments du système d'appel SysV. Lors de l'écriture des arguments dans ces registres, j'utilise la version 64 bits, mais pour la lecture ou l'écriture des valeurs, je sélectionne dynamiquement le registre en fonction du type de la variable (int, char).

2. Génération de labels uniques :

Un compteur global lblcnt me permet de générer des étiquettes uniques

pour les blocs de contrôle (if, while, expressions booléennes...), afin d'éviter toute collision entre labels dans l'assembleur.

3. Chargement correct des types signés :

Dans la fonction **load_var**, j'utilise **movsx** pour les char et **movsxd** pour les int, ce qui permet d'assurer une extension correcte des entiers signés vers 64 bits et de préserver les signes lors des calculs.

4. Traitement des caractères spéciaux :

J'ai mis en place une fonction **char_literal_value** permettant de traiter les caractères spéciaux (\n, \t, etc.), ainsi que les notations en octal et en hexadécimal. J'ai également ajouté le support explicite de \r, car son absence provoquait des erreurs répétées sous Windows lors de l'exécution des tests.

5. Évaluation logique normalisée :

Toutes les opérations booléennes et logiques sont traduites de manière à produire strictement 0 ou 1 comme résultat, ce qui garantit une cohérence avec le comportement attendu du langage C.

6. Allocation du cadre de pile :

Avant la génération de code pour un corps de fonction, j'effectue un scan complet de ses variables locales afin de calculer la taille totale nécessaire pour la pile (frameBytes), ce qui permet d'allouer l'espace d'un seul coup, en tête de fonction.

7. Retour implicite dans main :

Si la fonction **main** ne contient pas de **return**, je m'assure qu'un **mov rax, 0** est automatiquement ajouté à la fin, conformément aux règles du langage C.

8. Gestion du tampon d'entrée (clrbuf) :

J'ai implémenté une fonction intégrée **clrbuf** pour vider la ligne après un appel à **getchar**. Sans cela, un **getchar** suivi d'un **getint** capturait directement le caractère `\n` restant, ce qui faussait les lectures. Cette correction s'est révélée indispensable pour faire fonctionner les entrées utilisateur de manière fiable.

D'un point de vue personnel, ce qui m'a paru le plus difficile dans cette partie était de bien comprendre ce que chaque portion du code devait produire exactement, et surtout comment chaque cas (opérations, appels, branches...) devait être traduit en séquences assembleur précises. Contrairement à la génération de la table des symboles, ici il ne suffit pas de parcourir l'arbre : il faut reproduire fidèlement le comportement du programme, ce qui impose une parfaite compréhension de la structure syntaxique et sémantique, sans redondance ni omission dans les parcours.

Bien que j'aie couvert la majorité des cas pendant le développement, il me reste encore quatre tests qui échouent dans la validation automatique. Je pense que ce ne sont pas des oublis de nœuds à traiter, mais plutôt des cas particuliers ou des combinaisons rares que je n'ai pas anticipés.

5. Tests et validation

J'estime avoir couvert la grande majorité des cas possibles dans mes jeux de tests. Cela dit, comme j'ai finalisé l'écriture des tests après avoir terminé le code, certaines catégories comme `sem-err`, `syn-err` et `warn` ont pu être construites facilement en se basant directement sur la logique de détection dans le compilateur. En revanche, les tests de type `good` ont dû être rédigés à partir de cas imaginés, ce qui implique une certaine incertitude.

Il est donc possible que certains tests `good` incluent des constructions spécifiques que mon compilateur ne traite pas encore parfaitement, en

particulier si le problème n'est pas détectable uniquement via la valeur de retour. Lors de l'écriture de ces tests, je pensais sincèrement qu'ils étaient corrects — et d'un point de vue fonctionnel, ils le sont : les résultats retournés sont bien les bons. Toutefois, le système d'autotests que j'ai mis en place ne vérifie pas la sortie standard, ce qui peut masquer certains défauts dans la génération de code.

Un autre problème que j'ai rencontré concerne la compatibilité des fichiers sous Windows. En effet, à cause des caractères de fin de ligne `\r\n`, tous mes premiers tests automatiques échouaient avec un score de 40/100. Il m'a fallu du temps (au moins une heure) pour comprendre que ce n'était pas un bug lié à une mauvaise gestion des types ou à des erreurs de logique dans les tests — mais simplement un souci de format de fichiers. Après avoir passé tous les fichiers `.tpc` au filtre `dos2unix`, les tests ont fonctionné comme prévu.

6. Quelques mots personnels

Tout comme pour le projet d'analyse syntaxique du semestre précédent, j'ai réalisé ce projet entièrement seul. Je pense avoir donné le meilleur de moi-même : c'est un travail d'envergure, et au moment d'écrire ce rapport, j'ai déjà oublié une partie des détails de ce que j'ai réellement fait — et je suis certain que dans quelques jours, j'en aurai oublié encore plus.

Cela dit, je ne cherche pas à me plaindre : c'est un choix que j'ai fait volontairement, et je l'assume pleinement. Malgré la charge importante, j'ai pris du plaisir à développer ce compilateur et j'ai appris beaucoup de choses au passage. Apprendre en autonomie est aussi, selon moi, une compétence essentielle.

J'ai particulièrement pris conscience de l'importance des jeux de tests. C'est un aspect qui reflète sans doute ce qu'on rencontre en entreprise : avec un jeu de tests complet dès le départ, le projet aurait été beaucoup plus fluide à réaliser. Concernant les erreurs sémantiques, j'ai effectivement obtenu un score parfait aux tests automatiques, mais beaucoup de cas n'ont été couverts que **grâce à un hasard heureux**, lors d'un moment de repos par exemple. Pour la génération NASM, je n'ai pas eu cette chance...

Pour conclure, je tiens à remercier tous les enseignants pour leur investissement tout au long de ce semestre — ou même des semestres précédents. Il est possible que vous m'ayez eu en cours à différents moments. Même si je suis resté discret en classe (principalement à cause de la langue), j'ai toujours écouté avec attention. J'espère sincèrement pouvoir valider ce dernier semestre en France dans les meilleures conditions.

Si vous avez la moindre question concernant ce projet, n'hésitez pas à me contacter.

Et enfin, merci d'avoir pris le temps de lire ce rapport.